

Lecture 2

Training Deep Neural Networks

Advanced Topics in Artificial Intelligence · UFABC

Part 1 — Application

A real-world problem as a running example: predicting **heart disease**.

Application: heart disease prediction

- **Cleveland Clinic Heart Disease** dataset (~300 patients)
- 13 clinical attributes (age, sex, cholesterol, ECG, ...) → after *one-hot encoding*: **29 inputs**
- Binary output: was the patient diagnosed?
- **Network:** 1 hidden layer (16 ReLU) + sigmoid output

patient	age	sex	chol.	...	disease
1	63	M	233	...	0
2	67	M	286	...	1
3	67	M	229	...	1
4	37	M	250	...	0
5	41	F	204	...	0

$$\underbrace{29 \times 16}_{W^{(1)}} + \underbrace{16}_{b^{(1)}} + \underbrace{16 \times 1}_{W^{(2)}} + \underbrace{1}_{b^{(2)}} = \mathbf{497} \text{ parameters}$$

In code — Keras vs PyTorch

Keras / TensorFlow

```
from tensorflow import keras

inp = keras.Input(shape=(29,))
h = keras.layers.Dense(16, 'relu')(inp)
out = keras.layers.Dense(1, 'sigmoid')(h)
model = keras.Model(inp, out)

model.compile(
    loss='binary_crossentropy',
    optimizer='adam',
)
model.summary()
```

Keras: declares architecture + `compile` configures loss and optimizer.

PyTorch

```
import torch.nn as nn
import torch.optim as optim

model = nn.Sequential(
    nn.Linear(29, 16),
    nn.ReLU(),
    nn.Linear(16, 1),
    nn.Sigmoid(),
)

criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters())
print(model)
```

PyTorch: declares architecture + instantiates loss and optimizer separately.

Part 2 — Training = Minimizing the *loss*

Finding weights that make the network predict well is an **optimization** problem.

Loss function

- Measures the **error** between prediction and ground truth
- Good predictions → **small** loss; perfect model → **zero** loss
- The loss must **match the output type**:

Continuous output

→ MSE

Binary probability

→ BCE

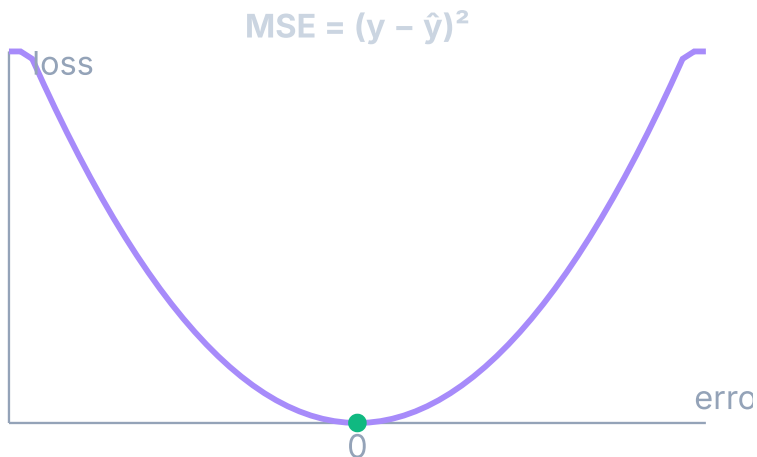
Multiclass

→ Cross-Entropy

MSE and Binary Cross-Entropy

MSE — numeric outputs:

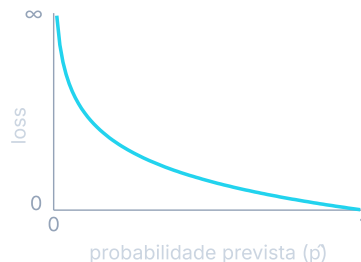
$$\mathcal{L}_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$



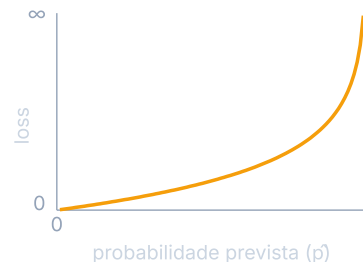
BCE — binary probability:

$$\ell_i = -y_i \log \hat{p}_i - (1 - y_i) \log(1 - \hat{p}_i)$$

Loss para $y = 1$



Loss para $y = 0$



When $y = 1$: penalizes low $\hat{p}$ ($-\log \hat{p} \rightarrow \infty$). When $y = 0$: penalizes high $\hat{p}$.

Part 3 — Gradient Descent

How do you minimize a function with thousands of variables?

Gradient descent algorithm

The derivative $g'(w)$ indicates the **local slope** — the direction in which g grows. Moving in the opposite direction leads us to the minimum:

$$w \leftarrow w - \alpha \cdot g'(w)$$

sign of $g'(w)$	action
> 0	decrease w
< 0	increase w
≈ 0	possible minimum

α = **learning rate** (typically 10^{-3} to 10^{-1}). Idea by Cauchy, 1847 — the same algorithm that trains GPT.



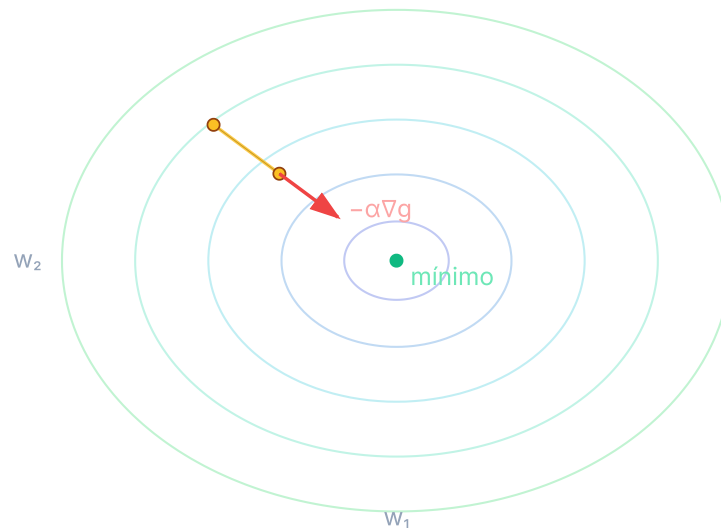
Multivariate case and local minima

For $\mathbf{w} \in \mathbb{R}^p$, the derivative becomes the **gradient**:

$$\nabla \mathcal{L}(\mathbf{w}) = \left[\frac{\partial \mathcal{L}}{\partial w_1}, \dots, \frac{\partial \mathcal{L}}{\partial w_p} \right]$$

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla \mathcal{L}(\mathbf{w})$$

In non-convex landscapes, GD can get stuck in **local minima** or **saddle points**. In practice, large networks have so many parameters that "bad" minima are rare.



A trajetória cruza as curvas de nível em direção ao centro.

Part 4 — Backpropagation

How do we compute $\nabla \mathcal{L}$ efficiently for **millions** of parameters?

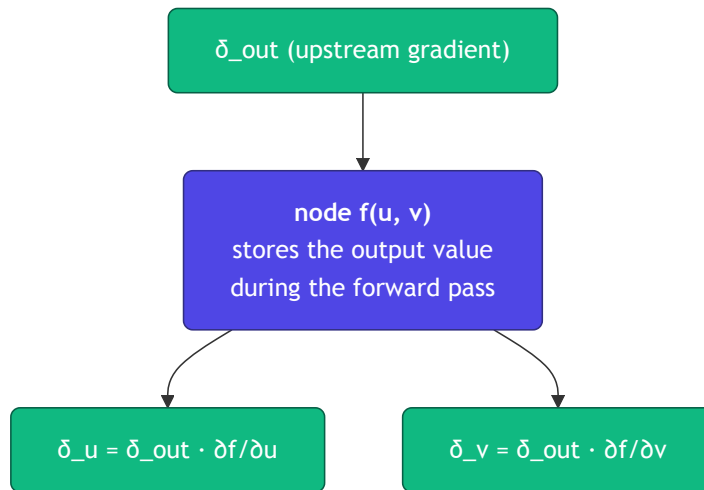
The problem

- For GD, we need $\nabla \mathcal{L}(\mathbf{w})$ — **one partial derivative per parameter**
- Computing each derivative independently repeats calculations — **infeasible** for large networks
- Core idea: organize the network's operations as a **computational graph** and apply the **chain rule** backwards

Backpropagation = chain rule + reuse of intermediate results.

Computational graph — anatomy of a node

- Each **operation** in the network (addition, multiplication, activation) is a **node** in the graph
- During the **forward pass**, each node stores its output value
- During the **backward pass**, each node must compute two types of gradient:
 - **local gradient:**
 $\partial(\text{node output})/\partial(\text{node input})$
 - **global gradient:** passes along the gradient coming from the next node

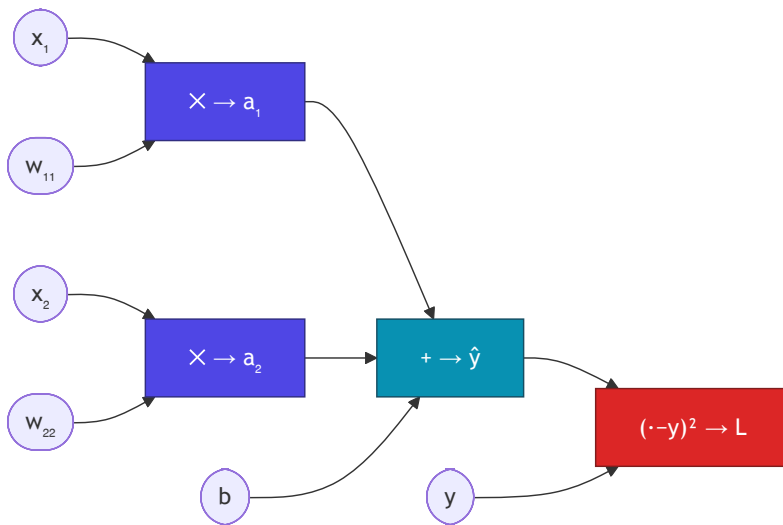


Each node multiplies the incoming gradient (δ_{out}) by its **local gradient** ($\partial f / \partial u, \partial f / \partial v$) and propagates it backwards.

Graph of our toy network

Network: 2 inputs, 1 linear hidden layer, 1 output, MSE loss:

$$a_1 = w_{11} x_1 \quad a_2 = w_{22} x_2 \quad \hat{y} = b + a_1 + a_2 \quad \mathcal{L} = (\hat{y} - y)^2$$



Trainable parameters: w_{11} , w_{22} , b . Inputs x_1 , x_2 and label y are **fixed** in the data.

Forward pass — numerical example

Values for one example: $x_1 = 2$, $x_2 = 3$, $w_{11} = 0.5$, $w_{22} = -1$, $b = 1$, $y = 4$.

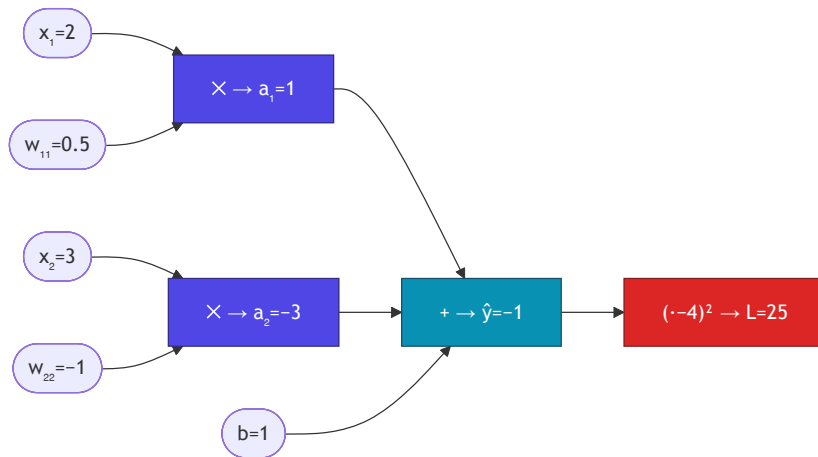
Left-to-right traversal:

$$a_1 = w_{11} \cdot x_1 = 0.5 \times 2 = \mathbf{1}$$

$$a_2 = w_{22} \cdot x_2 = -1 \times 3 = \mathbf{-3}$$

$$\hat{y} = b + a_1 + a_2 = 1 + 1 + (-3) = \mathbf{-1}$$

$$\mathcal{L} = (\hat{y} - y)^2 = (-1 - 4)^2 = \mathbf{25}$$



Local gradients at each node

For each node, we compute the derivative of **its output** with respect to **each of its inputs**:

Node	Operation	Local gradient
mul1	$a_1 = w_{11} \cdot x_1$	$\partial a_1 / \partial w_{11} = x_1 = 2$ $\partial a_1 / \partial x_1 = w_{11} = 0.5$
mul2	$a_2 = w_{22} \cdot x_2$	$\partial a_2 / \partial w_{22} = x_2 = 3$ $\partial a_2 / \partial x_2 = w_{22} = -1$
add	$\hat{y} = b + a_1 + a_2$	$\partial \hat{y} / \partial b = 1$ $\partial \hat{y} / \partial a_1 = 1$ $\partial \hat{y} / \partial a_2 = 1$
sq	$\mathcal{L} = (\hat{y} - y)^2$	$\partial \mathcal{L} / \partial \hat{y} = 2(\hat{y} - y) = 2(-1 - 4) = -10$

Each local gradient depends only on the **values stored during the forward pass** — that is why we keep everything.

Backward pass — propagating the gradients

We multiply the **upstream gradient** by the **local gradient**, right to left:

Node	Accumulated gradient
<code>sq</code> $\rightarrow \hat{y}$	$2(\hat{y} - y) = -10$
<code>add</code> $\rightarrow b$	$-10 \times 1 = -10$
<code>add</code> $\rightarrow a_1$	$-10 \times 1 = -10$
<code>add</code> $\rightarrow a_2$	$-10 \times 1 = -10$
<code>mul1</code> $\rightarrow w_{11}$	$-10 \times 2 = -20$
<code>mul2</code> $\rightarrow w_{22}$	$-10 \times 3 = -30$

$$\frac{\partial \mathcal{L}}{\partial w_{11}} = -20$$

$$\frac{\partial \mathcal{L}}{\partial w_{22}} = -30$$

$$\frac{\partial \mathcal{L}}{\partial b} = -10$$

Verification: chain rule "by hand"

We can confirm with the expanded formula:

$$\mathcal{L} = (b + w_{11}x_1 + w_{22}x_2 - y)^2$$

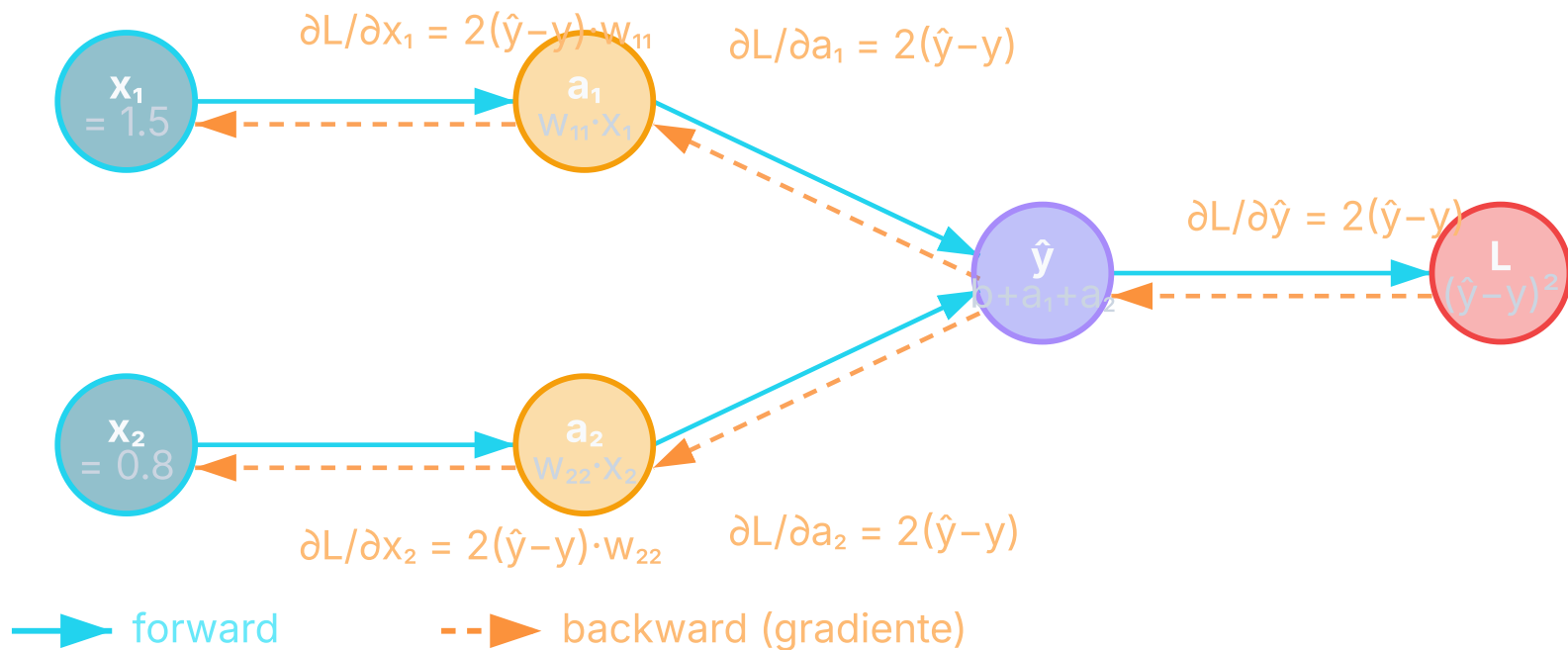
$$\frac{\partial \mathcal{L}}{\partial w_{11}} = 2(\hat{y} - y) \cdot x_1 = 2(-5) \cdot 2 = -\mathbf{20} \checkmark$$

Backprop gives the **same answer**, but with a crucial advantage:

- We **do not expand** the full expression
- Each node contributes with **one simple multiplication**
- Local gradients are **reused** by all paths in the graph
- In deep networks, this saves **billions of operations**

In practice, TensorFlow/PyTorch build and traverse this graph automatically — you only write the forward pass.

Forward + backward pass — visualization



Forward (blue) computes and stores values. Backward (orange) multiplies local $\times$ upstream gradients, layer by layer.

Automatic differentiation (*autograd*)

- Implementing backprop by hand is infeasible for real networks
- Frameworks build the **computational graph** **during the forward pass** and compute all gradients automatically
- Just write the forward — the backward is free
- In **PyTorch**: `loss.backward()` triggers autograd
- In **Keras**: `model.fit()` calls `tf.GradientTape` internally

```
import torch

# same values as the previous example
x = torch.tensor([2.0], requires_grad=True)
w = torch.tensor([0.5], requires_grad=True)
b = torch.tensor([1.0], requires_grad=True)
y_real = torch.tensor([4.0])

# forward — graph is built here
a = w * x
yh = b + a
loss = (yh - y_real) ** 2

# backward — traverses the graph automatically
loss.backward()

print(w.grad)    # ∂loss/∂w → tensor([-20.])
print(b.grad)    # ∂loss/∂b → tensor([-10.])
```

The gradients match what we computed on paper. ✓

Updating the weights

With the gradients in hand, GD updates **all parameters** simultaneously:

$$w_{11} \leftarrow 0.5 - \alpha \cdot (-20) = 0.5 + 20\alpha$$

$$w_{22} \leftarrow -1 - \alpha \cdot (-30) = -1 + 30\alpha$$

$$b \leftarrow 1 - \alpha \cdot (-10) = 1 + 10\alpha$$

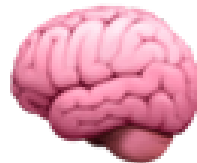
The negative gradients cause the weights to **increase** — which makes sense, since $\hat{y} = -1$ is far below $y = 4$. In a real network, we repeat this cycle for **millions of weights, per minibatch, per epoch**.

Why GPUs?

- The central step of backprop is **matrix multiplication** (gradient of a Dense layer)
- GPUs were created for gaming: they excel at **parallel matrix operations**
- Backprop + GPU = gradient of a huge network in **seconds**
- Today TPUs and dedicated accelerators (Google, Apple, NVIDIA) take the idea even further



GPU for gaming...



...engine of deep learning

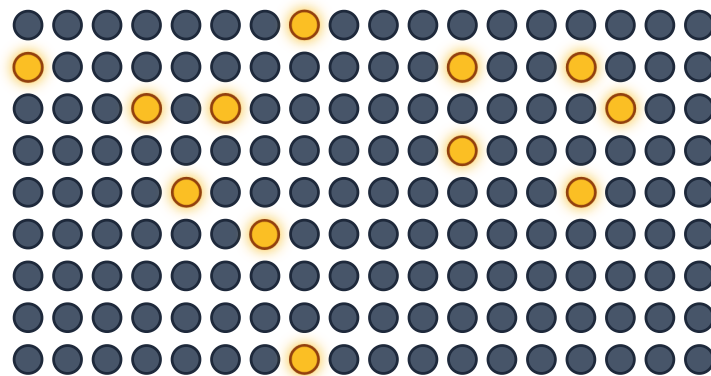
Part 5 — SGD and Optimizers

What happens when the dataset has millions of examples?

Mini-batches and SGD

- The loss is a **sum** over all n examples — computing the exact gradient requires going through everything
- Solution: at each iteration, **randomly** select a subset of ~32–256 examples (**minibatch**)
- The gradient estimated on the minibatch is a valid **approximation** — and works very well
- The introduced noise can even **help** escape bad minima
- This is **SGD** (Stochastic Gradient Descent)

12 de 162 exemplos sorteados



↓
 ∇ aproximado

A cada passo, sortamos um **minibatch** diferente.

Epochs, batches, and optimizers

Epoch = 1 complete pass through the training set.

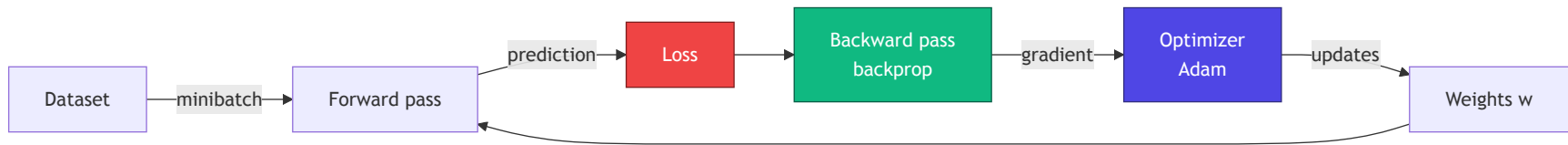
$$\text{batches/epoch} = \left\lceil \frac{n}{\text{batch size}} \right\rceil$$

Example — heart disease: 194 examples, batch 32 → 7 batches per epoch (6×32 + 2 = 194).

Adam — the default optimizer in DL:

- Adds **momentum** (inertia in consistent directions)
- **Adaptive learning rates** per parameter
- Bias correction in the first steps
- In Keras: `optimizer='adam'` and done

Complete training flow

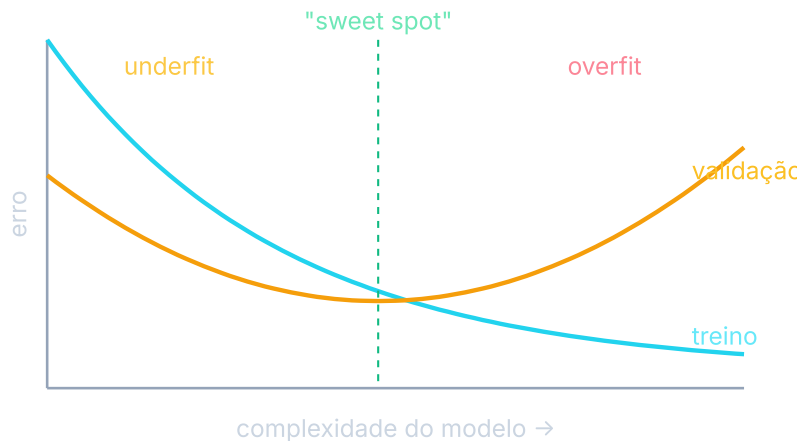


Each loop iteration = one **optimization step**. Multiple passes through the dataset = **epochs**.

Part 6 — Training in Practice

Overfitting, regularization, and the complete checklist.

Underfitting × Overfitting



Underfitting

Insufficient capacity. High error on both training *and* validation.

Overfitting

Memorizes the training set (including noise). Low training error, high validation error.

Regularization: Early Stopping and Dropout

Early Stopping

- Monitor the **validation** loss at each epoch
- Stop when it stops decreasing (even if the training loss keeps falling)
- Save the weights from the **best** moment

```
es = keras.callbacks.EarlyStopping(  
    patience=10,  
    restore_best_weights=True  
)
```

Dropout

- At each training step, randomly "switches off" a fraction p of the neurons
- Forces the network to **distribute** learning
- Disabled during inference (validation/test)

```
keras.layers.Dropout(0.5) # switches off 50% per step
```

Full training — Keras

```
# architecture
inp = keras.Input(shape=(29,))
h   = keras.layers.Dense(16, activation='relu')(inp)
h   = keras.layers.Dropout(0.3)(h)
out = keras.layers.Dense(1, activation='sigmoid')(h)
model = keras.Model(inp, out)

# compile
model.compile(
    loss='binary_crossentropy',
    optimizer='adam',
    metrics=['accuracy'],
)

# train
es = keras.callbacks.EarlyStopping(patience=10, restore_best_weights=True)
hist = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=200, batch_size=32,
    callbacks=[es],
)
```

Full training — PyTorch

```
# same architecture (nn.Sequential equivalent to the previous slide)
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters())

best_val, patience, wait, best_w = float('inf'), 10, 0, None
for epoch in range(200):
    model.train()
    optimizer.zero_grad()
    criterion(model(X_train), y_train).backward()
    optimizer.step()
    model.eval()
    with torch.no_grad():
        val = criterion(model(X_val), y_val).item()
    if val < best_val: best_val, wait, best_w = val, 0, model.state_dict().copy()
    elif (wait := wait + 1) >= patience: break
model.load_state_dict(best_w)
```

Explicit loop: `zero_grad` → `forward` → `backward` → `step` . More verbose than `model.fit()` , but makes every operation visible.

Checklist for training a DNN

1. **Prepare data** — encoding, normalization, train/val/test split
2. **Design the network** — number of layers, neurons, activations
3. **Choose the output** appropriate to the problem (sigmoid, softmax, linear)
4. **Choose the loss** that matches the output
5. **Optimizer** (Adam) + learning rate
6. **Regularization** (early stopping + dropout)
7. **Compile and train** with Keras or write the loop in PyTorch
8. **Evaluate** on validation and, at the end, on test

Summary

Loss — MSE (regression), BCE (binary), Cross-Entropy (multiclass)

Gradient Descent — moves opposite to the gradient, step α (learning rate)

Computational graph — each node stores value + local gradient

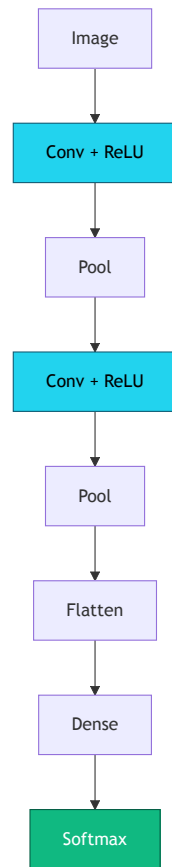
Backprop — multiplies upstream $\times$ local gradient, right to left

SGD / Adam — minibatch + momentum + adaptive learning rate

Early stopping + Dropout — essential regularization against overfitting

Next lecture

- **Tensors and images** — how to represent pixels in N-dimensional arrays
- **Why dense layers are not enough** — too many parameters, no spatial structure
- **Convolutional filters** — edge, texture, and visual pattern detection
- **Pooling** — dimensionality reduction while preserving information
- **CNNs** — Conv + Pool + FC blocks
- **Transfer Learning** — reusing pre-trained models (ResNet, ImageNet)



Thank you! Questions?

Freely adapted from *15.773 Hands-on Deep Learning — Lectures 02 & 03* (MIT OpenCourseWare, 2024) — original English material by Vivek Farias and Rama Ramakrishnan, distributed under the terms of MIT OCW.

For more information: <https://ocw.mit.edu/terms>